



Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Bảo vệ luận văn Thạc sĩ — Ngành Công nghệ Thông tin

Học viên: **Bùi Văn Dũng** — MSHV 220201014

TP. Hồ Chí Minh, 2026

Cán bộ hướng dẫn: **TS. Nguyễn Văn Dũ** · **TS. Đỗ Trí Nhựt**

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT



I. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU NGHIÊN CỨU

II. PHƯƠNG PHÁP THỰC HIỆN

III. KẾT QUẢ 1 — ĐÁNH GIÁ OFFLINE LOẠI TRỪ RÒ RỈ

IV. KẾT QUẢ 2 — TRIỂN KHAI PHÂN TÁN TRÊN CỤM BIÊN

V. TRẢ LỜI CÂU HỎI NGHIÊN CỨU VÀ KẾT LUẬN

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

└─Nội dung trình bày

Hai nhánh kết quả (III và IV) mỗi nhánh đã công bố thành một bài báo — FAIR 2026 cho phần offline, SOICT 2026 cho phần triển khai biên; slide Công trình khoa học ở cuối có chi tiết. Nói miệng ở đây, không cần chiếu.

- I. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU NGHIÊN CỨU
- II. PHƯƠNG PHÁP THỰC HIỆN
- III. KẾT QUẢ 1 — ĐÁNH GIÁ OFFLINE LOẠI TRỪ RÒ RỈ
- IV. KẾT QUẢ 2 — TRIỂN KHAI PHÂN TÁN TRÊN CỤM BIÊN
- V. TRẢ LỜI CÂU HỎI NGHIÊN CỨU VÀ KẾT LUẬN

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

I. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU NGHIÊN CỨU

I. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU
NGHIÊN CỨU

I. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU NGHIÊN CỨU

Vấn đề thực tiễn

- Thiết bị IoT nhiều, tài nguyên ít, bảo mật yếu.
- Lưu lượng lớn → cần nền tảng **phân tán**.
- Cần phát hiện **ngay tại biên** để giảm độ trễ.

Ba khoảng trống

1. RF, SHAP, PCA chưa được so sánh trên *cùng một* pipeline.
2. Điểm số CICIDS2017 thường **gần như hoàn hảo** — dấu hiệu rò rỉ.
3. IDS biên hầu như chỉ đo trên **một thiết bị**.

Cách tiếp cận: hai pha bổ trợ

(A) Offline — chọn mô hình nào, bao nhiêu đặc trưng.

(B) Biên — mô hình đó có chạy thời gian thực được không?

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

I. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU NGHIÊN CỨU

Bối cảnh và ba khoảng trống nghiên cứu

Nhấn mạnh: hai pha trả lời hai câu hỏi khác nhau, không phải hai phương án cạnh tranh. Pha A dưới quy trình loại trừ rõ rĩ. Ba câu hỏi nghiên cứu cụ thể ở slide ngay sau.

Vấn đề thực tiễn

- Thiết bị IoT nhiều, tài nguyên ít, bảo mật yếu.
- Lưu lượng lớn → cần nền tảng **phân tán**.
- Cần phát hiện **ngay tại biên** để giảm độ trễ.

Ba khoảng trống

1. RF, SHAP, PCA chưa được so sánh trên cùng một pipeline.
2. Điểm số CICIDS2017 thường **gần như hoàn hảo** — dấu hiệu rò rỉ.
3. IDS biên hầu như chỉ đo trên **một thiết bị**.

Cách tiếp cận: hai pha bổ trợ

(A) Offline — chọn mô hình nào, bao nhiêu đặc trưng.

(B) Biên — mô hình đó có chạy thời gian thực được không?

Mục tiêu và ba câu hỏi nghiên cứu

Mục tiêu. Xây dựng hệ thống IDS trên Apache Spark, đánh giá dưới quy trình loại trừ rò rỉ dữ liệu và kiểm chứng khả năng chạy thời gian thực trên cụm thiết bị biên.

RQ1 — Giảm chiều

Cắt hơn 60% đặc trưng có giữ được hiệu năng không?

RQ2 — Học tổ hợp

Có hơn mô hình đơn tốt nhất một cách đáng kể không?

RQ3 — Khả thi ở biên

Chạy được thời gian thực trên cụm Jetson không?

Phạm vi. CICIDS2017, phân loại nhị phân, cụm 2 Jetson Orin Nano Super. Bộ dữ liệu không có lưu lượng IoT đặc thù — tính “IoT” nằm ở kịch bản triển khai biên

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

I. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU NGHIÊN CỨU

Mục tiêu và ba câu hỏi nghiên cứu

Đọc chậm ba RQ; hội đồng sẽ đối chiếu với phần kết luận. RQ2 được kiểm chứng bằng kiểm định thống kê chứ không so một lần đo. Nêu chủ động giới hạn phạm vi IoT và tấn công đối kháng nằm ngoài phạm vi — slide Q1 có bản đầy đủ kèm mô hình mối đe dọa.

Mục tiêu và ba câu hỏi nghiên cứu		
Mục tiêu. Xây dựng hệ thống IDS trên Apache Spark, đánh giá dưới quy trình loại trừ rò rỉ dữ liệu, và kiểm chứng khả năng chạy thời gian thực trên cụm thiết bị biên.		
RQ1 — Giảm chiều Cắt hơn 60% đặc trưng có giữ được hiệu năng không?	RQ2 — Học tổ hợp Có hơn mô hình đơn tốt nhất một cách đáng kể không?	RQ3 — Khả thi ở biên Chạy được thời gian thực trên cụm Jetson không?
Phạm vi. CICIDS2017, phân loại nhị phân, cụm 2 Jetson Orin Nano Super. Bộ dữ liệu không có lưu lượng IoT đặc thù — tính “IoT” nằm ở kịch bản triển khai biên.		

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

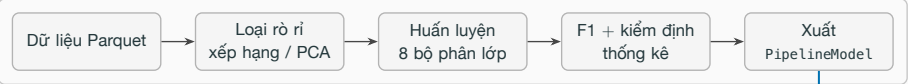
└ II. PHƯƠNG PHÁP THỰC HIỆN

II. PHƯƠNG PHÁP THỰC HIỆN

II. PHƯƠNG PHÁP THỰC HIỆN

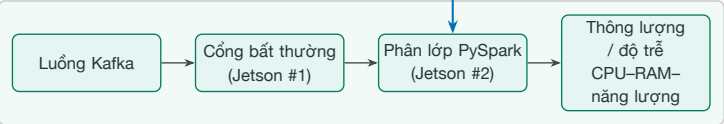
Kiến trúc tổng thể: hai pha của cùng một pipeline

PHA OFFLINE — cụm Spark (máy chủ master + 2 Jetson worker)



xuất mô hình đã kiểm định

PHA BIÊN — 2× Jetson Orin Nano Super



Hai pha không cạnh tranh nhau: pha offline trả lời “mô hình nào, bao nhiêu đặc trưng”; pha biên trả lời “mô hình đó có chạy được theo thời gian thực dưới ràng buộc bộ nhớ, nhiệt và băng thông hay không”

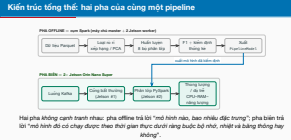
2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

II. PHƯƠNG PHÁP THỰC HIỆN

Kiến trúc tổng thể: hai pha của cùng một pipeline

Đây là slide bản lề của toàn bộ bài trình bày. Chỉ vào mũi tên xanh: mô hình được triển khai chính là mô hình đã qua kiểm định offline.



Hai nguồn rò rỉ, xử lý *trước khi chia tập*

- 1. **Đặc trưng rò rỉ nhãn:**
destination_port mã hóa thẳng dịch vụ bị tấn công.
- 2. **Trùng lặp mức đặc trưng:** cùng một mẫu lọt vào cả tập huấn luyện lẫn tập kiểm tra.

Nhóm nhận xung đột bị **loại cả nhóm**: 270 nhóm, 44.260 dòng. Còn 77 đặc trưng.

Bốn chiến lược, cùng một pipeline

- Baseline 77 đặc trưng · RF Top-K · SHAP Top-K · PCA $k=40$.
- 8 thuật toán + **Soft Voting**, **Hybrid Bagging**.

Chống thiên lệch: cấu hình cố định trước; tập kiểm tra **không tham gia** khâu lựa chọn nào.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

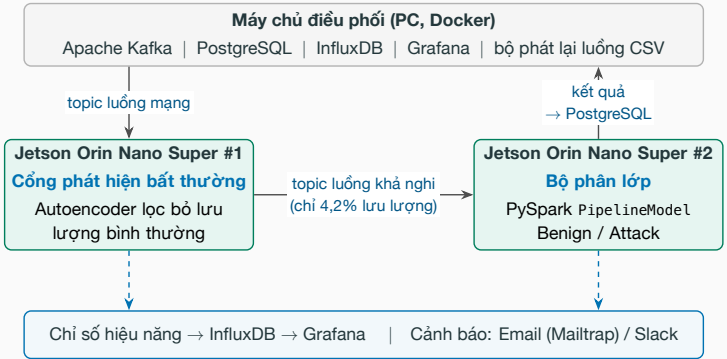
II. PHƯƠNG PHÁP THỰC HIỆN

Pha A — Quy trình loại trừ rò rỉ và bốn chiến lược so sánh

Đóng góp phương pháp luận quan trọng nhất. Nói rõ vì sao khử trùng lặp tất định lại quan trọng: kết quả không phụ thuộc cách Spark phân vùng dữ liệu. Xếp hạng RF/SHAP chỉ dùng tập huấn luyện; chọn mô hình trên tập kiểm định tách từ tập huấn luyện. PCA dùng $k=40$ vì là phép trích xuất — slide Q3 có lý do đầy đủ, kèm lý do loại LightGBM. Tám thuật toán: DT, LR, SVM, NB, RF, GBT, XGBoost, MLP.

Pha A — Quy trình loại trừ rò rỉ và bốn chiến lược so sánh	
Hai nguồn rò rỉ, xử lý <i>trước khi chia tập</i>	Bốn chiến lược, cùng một pipeline
1. Đặc trưng rò rỉ nhãn: destination_port mã hóa thẳng dịch vụ bị tấn công.	■ Baseline 77 đặc trưng · RF Top-K · SHAP Top-K · PCA $k=40$.
2. Trùng lặp mức đặc trưng: cùng một mẫu lọt vào cả tập huấn luyện lẫn tập kiểm tra.	■ 8 thuật toán + Soft Voting , Hybrid Bagging .
Chống thiên lệch: cấu hình cố định trước; tập kiểm tra không tham gia khâu lựa chọn nào.	
Nhóm nhận xung đột bị loại cả nhóm : 270 nhóm, 44.260 dòng. Còn 77 đặc trưng.	

Pha B — Kiến trúc triển khai phân tán trên cụm biên



Ba chế độ được đo: A tách pipeline (để xuất) - B mở rộng ngang - C cụm Spark; mốc tham chiếu là đơn nút. Thông lượng đếm **số luồng xử lý xong mỗi giây** — mỗi luồng chỉ tính một lần, tại nơi có kết luận.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

II. PHƯƠNG PHÁP THỰC HIỆN

Pha B — Kiến trúc triển khai phân tán trên cụm biên

Chế độ B: cả hai nút chạy vai trò đầy đủ. Ngưỡng MSE của cổng hiệu chỉnh ngoại tuyến, chốt trước khi đánh giá, không tinh chỉnh trên tập kiểm tra. Luồng chỉ tính một lần nên luồng đi qua cả hai nút không bị đếm đôi. Môi trường đo chi tiết ở slide Q8.



III. KẾT QUẢ 1 — ĐÁNH GIÁ OFFLINE LOẠI TRỪ RÒ RỈ

KQ1 — Giảm 60% đặc trưng mà vẫn giữ hiệu năng

Phương pháp	Mô hình	F1	Huấn luyện (s)	Đặc trưng
Baseline	XGBoost	0.9971	4615.5	77
SHAP Top-30	XGBoost	0,9970	1226,1	30
PCA k=40	XGBoost	0.9939	144.8	40
RF Top-30	XGBoost	0.9922	1295.1	30

Thuật toán	RF Top-30	SHAP Top-30	Chênh lệch
Random Forest	0.9879	0.9955	+0,77%
XGBoost	0.9922	0.9970	+0,48%
GBT	0.9844	0.9918	+0,75%
Decision Tree	0.9835	0.9948	+1,15%

- SHAP Top-30 giữ F1 ngang baseline, cắt **60% đặc trưng** và **73% thời gian huấn luyện**.
- Cùng số đặc trưng, SHAP vượt RF Importance trên *mọi* thuật toán họ cây.
- **Hybrid Bagging** + SHAP Top-30 đạt F1 **0,9966** — cao nhất.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

III. KẾT QUẢ 1 — ĐÁNH GIÁ OFFLINE LOẠI TRỪ RÒ RỈ

KQ1 — Giảm 60% đặc trưng mà vẫn giữ hiệu năng

Trả lời RQ1 ngay tại đây, nhưng chưa kết luận vội — còn phải kiểm định thống kê ở slide sau. Nói thêm: ít đặc trưng hơn thì chi phí VectorAssembler và bộ nhớ mô hình ở biên cũng nhỏ hơn; SHAP còn cho phép giải thích từng quyết định, có giá trị cho vận hành SOC.

KQ1 — Giảm 60% đặc trưng mà vẫn giữ hiệu năng

Phương pháp	Mô hình	F1	Huấn luyện (s)	Đặc trưng
Baseline	XGBoost	0.9971	4615.5	77
SHAP Top-30	XGBoost	0,9970	1226,1	30
PCA k=40	XGBoost	0.9939	144.8	40
RF Top-30	XGBoost	0.9922	1295,1	30

Thuật toán	RF Top-30	SHAP Top-30	Chênh lệch
Random Forest	0.9879	0.9955	+0,77%
XGBoost	0.9922	0.9970	+0,48%
GBT	0.9844	0.9918	+0,75%
Decision Tree	0.9835	0.9948	+1,15%

- SHAP Top-30 giữ F1 ngang baseline, cắt **60% đặc trưng** và **73% thời gian huấn luyện**.
- Cùng số đặc trưng, SHAP vượt RF Importance trên *mọi* thuật toán họ cây.
- **Hybrid Bagging** + SHAP Top-30 đạt F1 **0,9966** — cao nhất.

KQ2 — Kiểm định thống kê thay vì so sánh một lần đo

Phương pháp	Mô hình	F1 (TB ± CI95)
Baseline	XGBoost	0.99734 ± 0.00007
SHAP Top-30	XGBoost	0.99720 ± 0.00009
hoán vị ghép cặp $p = 0.031$; $d = 1$		
Ablation công (RF)	F1	Recall (Attack)
Giữ destination_port	0.9965	0.9988
Loại công (dễ xuất)	0.9955	0.9967

Chênh lệch tuyệt đối chỉ $\approx 0,0001 \rightarrow$ hai cấu hình **tương đương về mặt thực tiễn**; tiêu chí phụ mới quyết định.

- $N=6$ lần *lấy mẫu lại* ghép cặp, không chỉ đổi seed.
- Hoán vị hai phía **liệt kê đầy đủ** 2^6 dấu: p nhỏ nhất là $2/2^6 \rightarrow$ **bắt buộc $N \geq 6$** .
- Lấy mẫu *chống lẩn* $\rightarrow$ kết luận theo khoảng tin cậy **Nadeau–Bengio**.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

III. KẾT QUẢ 1 — ĐÁNH GIÁ OFFLINE LOẠI TRỪ RÒ RỈ

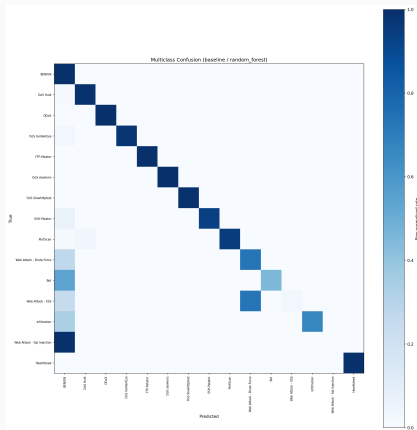
KQ2 — Kiểm định thống kê thay vì so sánh một lần đo

Đây là chỗ thể hiện sự cẩn trọng phương pháp: p nhỏ không đồng nghĩa khác biệt có ý nghĩa thực tiễn — ở đây $p = 0,031$ chỉ vừa chạm ngưỡng sàn của độ phân giải kiểm định. Lấy mẫu chống lẩn vi phạm giả định độc lập của phân phối t , nên Nadeau–Bengio là căn cứ chính.

KQ2 — Kiểm định thống kê thay vì so sánh một lần đo		
Phương pháp	Mô hình	F1 (TB ± CI95)
Baseline	XGBoost	0.99734 ± 0.00007
SHAP Top-30	XGBoost	0.99720 ± 0.00009
hoán vị ghép cặp $p = 0.031$; $d = 1$		
Ablation công (RF)	F1	Recall (Attack)
Giữ destination_port	0.9965	0.9988
Loại công (dễ xuất)	0.9955	0.9967
Chênh lệch tuyệt đối chỉ $\approx 0,0001 \rightarrow$ hai cấu hình <i>tương đương về mặt thực tiễn</i> ; tiêu chí phụ mới quyết định.		

- $N=6$ lần *lấy mẫu lại* ghép cặp, không chỉ đổi seed.
- Hoán vị hai phía **liệt kê đầy đủ** 2^6 dấu: p nhỏ nhất là $2/2^6 \rightarrow$ **bắt buộc $N \geq 6$** .
- Lấy mẫu *chống lẩn* $\rightarrow$ kết luận theo khoảng tin cậy **Nadeau–Bengio**.

KQ3 – Đánh giá đa lớp: F1 nhị phân che giấu điều quan trọng



Lớp	Số mẫu	Recall	F1
Benign	380.119	0,9997	0,9989
DoS Hulk	34.446	0,9904	0,9947
Bot	290	0,4552	0,6256
Web – XSS	111	0,0270	0,0526
Web – SQL Injection	6	0,0000	0,0000
weighted-F1 = 0,998		macro-F1 = 0,806	

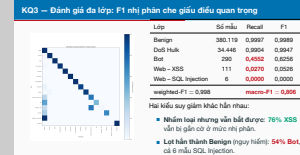
- **Nhầm loại nhưng vẫn bắt được: 76% XSS**
vẫn bị gán cờ ở mức nhị phân.
- **Lọt hỏn thành Benign (nguy hiểm): 54% Bot**,
cả 6 mẫu SQL Injection.

8-15 Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

III. KẾT QUẢ 1 – ĐÁNH GIÁ OFFLINE LOẠI TRỪ RÒ RỈ

KQ3 Đánh giá đo lường: E1 chỉ phân cho giáo viên điều quan

81/111 mẫu XSS bị đoán thành Web Brute Force, 27% Brute Force lọt thành Benign. Ba cơ chế: mất cân bằng cực đoan (6–311 mẫu so với 380 nghìn benign); loại destination_port tác động mạnh nhất lên web attack — mất lối tắt cổng 80, luồng HTTP ngán trông như duyệt web bình thường, nên recall 0,027 trung thực thay thế con số bị rò rỉ thổi phồng; và beaconing C&C của Bot vốn giống lưu lượng nền.



KQ4 — Tổng quát hóa liên bộ dữ liệu: con số trung thực nhất

Huấn luyện	Kiểm tra	F1
CICIDS2017	CICIDS2017	0,9952
CICIDS2017	CSE-CIC-IDS2018	0,0423
CSE-CIC-IDS2018	CSE-CIC-IDS2018	0,9245
CSE-CIC-IDS2018	CICIDS2017	0,0535

Tập đặc trưng chung đã loại bỏ rí; CSE-CIC-IDS2018 lấy mẫu phân tầng.

- Trong-miền cao ở cả hai chiều — liên bộ dữ liệu sụp xuống 0,04–0,05.
- Do recall gần 0, trong khi precision vẫn 0,596: mô hình không nhận ra tấn công của testbed kia.
- Nguyên nhân: khác phiên bản CICFlowMeter, khác cấu hình mạng, khác bộ công cụ tấn công.

Điểm số trong-miền gần hoàn hảo — kể cả khi đã loại rò rỉ — không chứng nhận khả năng triển khai dưới dịch chuyển phân phối.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

III. KẾT QUẢ 1 — ĐÁNH GIÁ OFFLINE LOẠI TRỪ RÒ RỈ

KQ4 — Tổng quát hóa liên bộ dữ liệu: con số trung thực nhất

Không làm nhẹ con số này. Recall cụ thể là 0,022 và 0,030; mẫu CSE-CIC-IDS2018 khoảng 2,39 triệu luồng, seed cố định. Kết quả thống nhất với các nghiên cứu liên bộ dữ liệu trước đó và là lý do thích ứng miền là điều kiện tiên quyết.

KQ4 — Tổng quát hóa liên bộ dữ liệu: con số trung thực nhất

Huấn luyện	Kiểm tra	F1
CICIDS2017	CICIDS2017	0,9952
CICIDS2017	CSE-CIC-IDS2018	0,0423
CSE-CIC-IDS2018	CSE-CIC-IDS2018	0,9245
CSE-CIC-IDS2018	CICIDS2017	0,0535

Tập đặc trưng chung đã loại bỏ rí; CSE-CIC-IDS2018 lấy mẫu phân tầng.

- Trong-miền cao ở cả hai chiều — liên bộ dữ liệu sụp xuống 0,04–0,05.
- Do recall gần 0, trong khi precision vẫn 0,596: mô hình không nhận ra tấn công của testbed kia.
- Nguyên nhân: khác phiên bản CICFlowMeter, khác cấu hình mạng, khác bộ công cụ tấn công.

Điểm số trong-miền gần hoàn hảo — kể cả khi đã loại rò rỉ — không chứng nhận khả năng triển khai dưới dịch chuyển phân phối.

IV. KẾT QUẢ 2 — TRIỂN KHAI PHÂN TÁN TRÊN CỤM BIÊN

KQ5 — Tách pipeline nâng thông lượng gấp 24 lần

Chỉ số	Thông lượng (luồng/giây)	p95 (ms)	Bỏ qua ở cổng (%)	Năng lượng (mJ/luồng)
Đơn nút	2.60 ± 0.41	12.3 ± 3.0	(nội tuyến)	2490
A: Tách pipeline	62,5 ± 0,6	13,1 ± 2,6	95,8	178
B: Mở rộng ngang	5.9 ± 1.4	21.6 ± 19.2	(nội tuyến)	2250
C: Cụm Spark	90.0 ± 25.6	23.9 ± 17.8	97.9	119

- **A: gấp 24 lần, p95 gần như không đổi** — cổng hấp thụ 95,8% lưu lượng bình thường.
- **B vẫn bị chặn bởi Spark** (≈ 2 lần) — lợi ích đến từ *tách vai trò*, không phải từ thêm board.

Lúc đỉnh tải, nút cổng chỉ dùng **4,6%** CPU so với **90,8%** ở nút phân lớp → tách pipeline là mặc định khuyến nghị.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

IV. KẾT QUẢ 2 — TRIỂN KHAI PHÂN TÁN TRÊN CỤM BIÊN

KQ5 — Tách pipeline nâng thông lượng gấp 24 lần

Bảng cốt lõi của pha B. Chỉ rõ chế độ B là bằng chứng phản chứng: thêm board mà không tách vai trò thì gần như không được gì. Chế độ C nhanh nhất về trung bình nhưng lệch ± 26 luồng/giây và phụ thuộc Spark master trên máy chủ. Năng lượng là mức toàn board trên mỗi luồng: ở tải này phần công suất tích cực nằm dưới ngưỡng nhiễu của tegrastats. Ảnh Grafana đỉnh tải ở slide Q11.

Chỉ số	Thông lượng (luồng/giây)	p95 (ms)	Bỏ qua ở cổng (%)	Năng lượng (mJ/luồng)
Đơn nút	2.60 ± 0.41	12.3 ± 3.0	(nội tuyến)	2490
A: Tách pipeline	62,5 ± 0,6	13,1 ± 2,6	95,8	178
B: Mở rộng ngang	5.9 ± 1.4	21.6 ± 19.2	(nội tuyến)	2250
C: Cụm Spark	90.0 ± 25.6	23.9 ± 17.8	97.9	119

■ **A: gấp 24 lần, p95 gần như không đổi** — cổng hấp thụ 95,8% lưu lượng bình thường.
■ **B vẫn bị chặn bởi Spark** (≈ 2 lần) — lợi ích đến từ tách vai trò, không phải từ thêm board.

Lúc đỉnh tải, nút cổng chỉ dùng **4,6%** CPU so với **90,8%** ở nút phân lớp → tách pipeline là mặc định khuyến nghị.

KQ7 — Cái giá trung thực của việc đồng nhất một stack

Phục vụ suy luận bằng Spark để *đồng nhất với pha huấn luyện phân tán*. Cùng mô hình, cùng 29 đặc trưng, cùng một board:

Engine suy luận	Thông lượng (đường/giây)	p95 (ms)	RAM (%)	Năng lượng (mJ/suy luận)
PySpark (đang triển khai)	22,6	532	24,3	71,6
scikit-learn	204,4	49,8	13,3	2,91
ONNX Runtime	7870,8	1,1	15,7	0,06

ONNX Runtime đạt thông lượng gấp **≈348 lần** bản PySpark đang triển khai. Luận văn báo cáo con số này như một *mục tiêu cho bước tiếp theo* — xuất mô hình sang ONNX/TensorRT cho pha triển khai biên, giữ Spark cho pha huấn luyện — chứ không phải lập luận bác bỏ stack đồng nhất.

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

IV. KẾT QUẢ 2 — TRIỂN KHAI PHÂN TÁN TRÊN CỤM BIÊN

KQ7 — Cái giá trung thực của việc đồng nhất một stack

Chủ động trình bày điểm yếu này. Hội đồng sẽ đánh giá cao việc tự định lượng chi phí của lựa chọn thiết kế thay vì che giấu.

KQ7 — Cái giá trung thực của việc đồng nhất một stack				
Phục vụ suy luận bằng Spark để đồng nhất với pha huấn luyện phân tán. Cùng mô hình, cùng 29 đặc trưng, cùng một board:				
Engine suy luận	Thông lượng (đường/giây)	p95 (ms)	RAM (%)	Năng lượng (mJ/suy luận)
PySpark (đang triển khai)	22,6	532	24,3	71,6
scikit-learn	204,4	49,8	13,3	2,91
ONNX Runtime	7870,8	1,1	15,7	0,06
ONNX Runtime đạt thông lượng gấp ≈348 lần bản PySpark đang triển khai. Luận văn báo cáo con số này như một mục tiêu cho bước tiếp theo — xuất mô hình sang ONNX/TensorRT cho pha triển khai biên, giữ Spark cho pha huấn luyện — chứ không phải lập luận bác bỏ stack đồng nhất.				

V. TRẢ LỜI CÂU HỎI NGHIÊN CỨU VÀ KẾT LUẬN

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

└ V. TRẢ LỜI CÂU HỎI NGHIÊN CỨU VÀ KẾT LUẬN

V. TRẢ LỜI CÂU HỎI NGHIÊN CỨU
VÀ KẾT LUẬN

RQ1 — Cắt hơn 60% đặc trưng có giữ được hiệu năng?

Có. SHAP Top-30 đạt F1 **0,9970** so với 0,9971 của baseline 77 đặc trưng, giảm $\approx 73\%$ thời gian huấn luyện.

RQ2 — Học tổ hợp có hơn mô hình đơn tốt nhất một cách đáng kể?

Không đáng kể về mặt thực tiễn. Hybrid Bagging cao nhất (F1 0,9966), nhưng chênh lệch tuyệt đối chỉ $\approx 0,0001$.

RQ3 — Pipeline Spark có khả thi trên cụm thiết bị biên?

Khả thi, với điều kiện tách vai trò. Thông lượng **2,6 $\rightarrow$ 62,5** luồng/giây ở p95 ≈ 13 ms; năng lượng **2,5 $\rightarrow$ 0,18 J** mỗi luồng.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

└ V. TRẢ LỜI CÂU HỎI NGHIÊN CỨU VÀ KẾT LUẬN

Trả lời trực tiếp ba câu hỏi nghiên cứu

Slide quan trọng nhất với hội đồng. Trả lời thẳng, kể cả câu trả lời “không” cho RQ2. RQ1: Random Forest đạt 0,9955; cùng số đặc trưng SHAP vượt RF Importance +0,48% đến +1,15% trên mọi thuật toán họ cây. RQ2: kiểm định hoán vị ghép cặp cho $p \approx 0,031$, chỉ vừa chạm ngưỡng sà — bài toán nhị phân trong-miền đã gần bão hòa, nên tiêu chí phụ mới là căn cứ chọn mô hình. RQ3: chỉ thêm board mà không tách vai trò (chế độ B) hầu như không cải thiện.

Trả lời trực tiếp ba câu hỏi nghiên cứu	
RQ1 — Cắt hơn 60% đặc trưng có giữ được hiệu năng?	Có. SHAP Top-30 đạt F1 0,9970 so với 0,9971 của baseline 77 đặc trưng, giảm $\approx 73\%$ thời gian huấn luyện.
RQ2 — Học tổ hợp có hơn mô hình đơn tốt nhất một cách đáng kể?	Không đáng kể về mặt thực tiễn. Hybrid Bagging cao nhất (F1 0,9966), nhưng chênh lệch tuyệt đối chỉ $\approx 0,0001$.
RQ3 — Pipeline Spark có khả thi trên cụm thiết bị biên?	Khả thi, với điều kiện tách vai trò. Thông lượng 2,6 $\rightarrow$ 62,5 luồng/giây ở p95 ≈ 13 ms; năng lượng 2,5 $\rightarrow$ 0,18 J mỗi luồng.

Về phương pháp luận

- **Quy trình đánh giá loại trừ rò rỉ**, khử trùng lặp *tất định* ở mức đặc trưng.
- **Kiểm định thống kê chặt chẽ**: hoán vị ghép cặp, khoảng tin cậy Nadeau–Bengio.
- **Đánh giá đa lớp và liên bộ dữ liệu** — chỉ ra điều F1 nhị phân che giấu.

Về hệ thống

- **Pipeline IDS hoàn chỉnh trên Apache Spark**, từ tiền xử lý đến suy luận thời gian thực.
- **Kiến trúc hai tầng trên cụm biên**: cổng bất thường + phân lớp PySpark, nối bằng Kafka.
- **Đo đặc vận hành đầy đủ**, kể cả năng lượng mỗi luồng.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

└ V. TRẢ LỜI CÂU HỎI NGHIÊN CỨU VÀ KẾT LUẬN

Đóng góp chính của luận văn

Chia rõ hai nhóm đóng góp: phương pháp luận và hệ thống. Bổ sung nếu được hỏi: quy trình có định lượng mức thời phỏng do rò rỉ; macro-F1 0,806 so với weighted-F1 0,998; ba chế độ triển khai đều được đo; mã nguồn và cấu hình công khai, tái lập được.

Đóng góp chính của luận văn

Về phương pháp luận

- Quy trình đánh giá loại trừ rò rỉ, khử trùng lặp *tất định* ở mức đặc trưng.
- Kiểm định thống kê chặt chẽ: hoán vị ghép cặp, khoảng tin cậy Nadeau–Bengio.
- Đánh giá đa lớp và liên bộ dữ liệu — chỉ ra điều F1 nhị phân che giấu.

Về hệ thống

- Pipeline IDS hoàn chỉnh trên Apache Spark, từ tiền xử lý đến suy luận thời gian thực.
- Kiến trúc hai tầng trên cụm biên: cổng bất thường + phân lớp PySpark, nối bằng Kafka.
- Đo đặc vận hành đầy đủ, kể cả năng lượng mỗi luồng.

Hạn chế

- **Phân loại:** triển khai chính là nhị phân.
- **Dữ liệu:** CICIDS2017 không có lưu lượng IoT đặc thù.
- **Quy mô:** mới ở mức *minh chứng khái niệm*, cụm 2 nút.
- **Đối kháng:** chưa kiểm thử lưu lượng né tránh; GPU còn bỏ trống.

Hướng phát triển — ứng với từng hạn chế

- **Lớp tấn công hiếm:** cân bằng có chủ đích, kiến trúc phân tầng.
- **Dữ liệu IoT chuyên biệt:** Bot-IoT, TON_IoT, CIC-IoT-2023.
- **ONNX/TensorRT**, khai thác GPU Jetson.
- **Bền vững đối kháng:** hai bề mặt mới do công tạo ra.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

— V. TRẢ LỜI CÂU HỎI NGHIÊN CỨU VÀ KẾT LUẬN

Hạn chế và hướng phát triển

Nêu hạn chế trước khi hội đồng nêu, mỗi hạn chế đi kèm hướng khắc phục tương ứng. Chi tiết: bộ phân lớp đa lớp thời gian thực chưa hoàn thiện; liên bộ dữ liệu mới thử trên mẫu phân tầng CSE-CIC-IDS2018; cụm 2 nút 8 GB chưa phải quy mô terabyte; GPU khoảng 67 TOPS chưa dùng vì pipeline chỉ chạy CPU. Bản hướng phát triển đầy đủ ở slide Q12.

Hạn chế và hướng phát triển

Hạn chế

- Phân loại: triển khai chính là nhị phân.
- Dữ liệu: CICIDS2017 không có lưu lượng IoT đặc thù.
- Quy mô: mới ở mức minh chứng khái niệm, cụm 2 nút.
- Đối kháng: chưa kiểm thử lưu lượng né tránh; GPU còn bỏ trống.

Hướng phát triển — ứng với từng hạn chế

- Lớp tấn công hiếm: cân bằng có chủ đích, kiến trúc phân tầng.
- Dữ liệu IoT chuyên biệt: Bot-IoT, TON_IoT, CIC-IoT-2023.
- ONNX/TensorRT, khai thác GPU Jetson.
- Bền vững đối kháng: hai bề mặt mới do công tạo ra.

1. Thai Nguyen Vu, **Dung Van Bui**, Nhut Tri Do, Van Du Nguyen, “*A Comparison of Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML with a Leakage-Aware Evaluation Pipeline*”, Hội thảo quốc gia Nghiên cứu cơ bản và ứng dụng CNTT (**FAIR**), 2026. **(Accepted)**

2. Thai Nguyen Vu, **Dung Van Bui**, Nhut Tri Do, Van Du Nguyen, “*A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark*”, Symposium on Information and Communication Technology (**SOICT**), 2026. **(Accepted)**

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

— V. TRẢ LỜI CÂU HỎI NGHIÊN CỨU VÀ KẾT LUẬN

Công trình khoa học từ luận văn

Công trình khoa học từ luận văn

1. Thai Nguyen Vu, **Dung Van Bui**, Nhut Tri Do, Van Du Nguyen, “*A Comparison of Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML with a Leakage-Aware Evaluation Pipeline*”, Hội thảo quốc gia Nghiên cứu cơ bản và ứng dụng CNTT (**FAIR**), 2026. **(Accepted)**

2. Thai Nguyen Vu, **Dung Van Bui**, Nhut Tri Do, Van Du Nguyen, “*A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark*”, Symposium on Information and Communication Technology (**SOICT**), 2026. **(Accepted)**

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

└ V. TRẢ LỜI CÂU HỎI NGHIÊN CỨU VÀ KẾT LUẬN

Xin trân trọng cảm ơn Hội đồng

Em xin lắng nghe các ý kiến nhận xét và câu hỏi.

Xin trân trọng cảm ơn Hội đồng

Em xin lắng nghe các ý kiến nhận xét và câu hỏi.

Bấm vào tên slide để nhảy thẳng tới số hiện. Click ở tiêu đề từng slide

Phạm vi và phương pháp

- Q1** Phạm vi “IoT”, mô hình mối đe dọa → **Phạm vi & đe dọa**
- Q2** Khử trùng lặp, chống thiên lệch → **Chi tiết tiền xử lý**
- Q3** Vì sao PCA $k=40$, vì sao loại LightGBM → **PCA & LightGBM**
- Q4** Siêu tham số từng mô hình → **Siêu tham số**

Kết quả offline

- Q5** So sánh đầy đủ 8 thuật toán $\times$ 4 phương pháp → **Bản đồ nhiệt F1**
- Q6** Độ tin cậy, giới hạn kết luận → **Đe dọa tính hợp lệ**

Triển khai biên

- Q7** Vì sao chọn Random Forest → **Chọn mô hình biên**
- Q8** Cấu hình cụm, ba chế độ đo → **Chế độ & môi trường**
- Q9** Cách đo thông lượng, độ trễ, năng lượng → **Giao thức đo**
- Q10** Ngưỡng cổng, đánh đổi recall → **Đường cong vận hành**
- Q11** Bảng chứng chạy thực tế → **Grafana đính tài**

Kết luận

- Q12** Kế hoạch tiếp theo, chi tiết → **Hướng phát triển**

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Q&A	
Bấm vào tên slide để nhảy thẳng tới số hiện. Click ở tiêu đề từng slide.	
Phạm vi và phương pháp	Triển khai biên
Q1 Phạm vi “IoT”, mô hình mối đe dọa → Phạm vi & đe dọa	Q7 Vì sao chọn Random Forest → Chọn mô hình biên
Q2 Khử trùng lặp, chống thiên lệch → Chi tiết tiền xử lý	Q8 Cấu hình cụm, ba chế độ đo → Chế độ & môi trường
Q3 Vì sao PCA $k=40$, vì sao loại LightGBM → PCA & LightGBM	Q9 Cách đo thông lượng, độ trễ, năng lượng → Giao thức đo
Q4 Siêu tham số từng mô hình → Siêu tham số	Q10 Ngưỡng cổng, đánh đổi recall → Đường cong vận hành
Kết quả offline	Q11 Bảng chứng chạy thực tế → Grafana đính tài
Q5 So sánh đầy đủ 8 thuật toán $\times$ 4 phương pháp → Bản đồ nhiệt F1	Kết luận
Q6 Độ tin cậy, giới hạn kết luận → Đe dọa tính hợp lệ	Q12 Kế hoạch tiếp theo, chi tiết → Hướng phát triển

Q1 — Phạm vi, dữ liệu và mô hình mối đe dọa

Phạm vi

- **Dữ liệu:** CICIDS2017; mở rộng sang CSE-CIC-IDS2018 cho thí nghiệm liên bộ dữ liệu.
- **Công nghệ:** Spark Core/SQL/Streaming/MLlib; Kafka, PostgreSQL, InfluxDB, Grafana.
- **Bài toán:** nhị phân Benign/Attack (có bổ sung đánh giá đa lớp theo loại tấn công).
- **Triển khai:** cụm 2 Jetson Orin Nano Super (8 GB) + máy chủ điều phối.

Lưu ý trung thực về phạm vi “IoT”

CICIDS2017 là lưu lượng doanh nghiệp *mô phỏng*, không chứa giao thức đặc thù IoT (MQTT, CoAP). Tính chất “IoT” nằm ở **kịch bản triển khai**: toàn bộ pipeline suy luận được đo trên *thiết bị biên tài nguyên hạn chế* — lớp gateway trong kiến trúc IoT ba tầng — chứ không ở bản chất lưu lượng huấn luyện.

Mô hình mối đe dọa

Kẻ tấn công ở bên trong hoặc ở biên mạng; phạm vi giới hạn ở các tấn công đã biết kiểu CICIDS2017. Tấn công đối kháng (adversarial/evasion) **nằm ngoài phạm vi** đánh giá.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Q1 — Phạm vi, dữ liệu và mô hình mối đe dọa

Chủ động nêu giới hạn phạm vi IoT trước khi hội đồng hỏi. Đây là câu hỏi phản biện gần như chắc chắn sẽ có.

Q1 — Phạm vi, dữ liệu và mô hình mối đe dọa		
Phạm vi	■ Dữ liệu: CICIDS2017; mở rộng sang CSE-CIC-IDS2018 cho thí nghiệm liên bộ dữ liệu.	Lưu ý trung thực về phạm vi “IoT” CICIDS2017 là lưu lượng doanh nghiệp mô phỏng, không chứa giao thức đặc thù IoT (MQTT, CoAP). Tính chất “IoT” nằm ở kịch bản triển khai : toàn bộ pipeline suy luận được đo trên <i>thiết bị biên tài nguyên hạn chế</i> — lớp gateway trong kiến trúc IoT ba tầng — chứ không ở bản chất lưu lượng huấn luyện.
	■ Công nghệ: Spark Core/SQL/Streaming/MLlib; Kafka, PostgreSQL, InfluxDB, Grafana.	
	■ Bài toán: nhị phân Benign/Attack (có bổ sung đánh giá đa lớp theo loại tấn công).	
■ Triển khai: cụm 2 Jetson Orin Nano Super (8 GB) + máy chủ điều phối.		Mô hình mối đe dọa Kẻ tấn công ở bên trong hoặc ở biên mạng; phạm vi giới hạn ở các tấn công đã biết kiểu CICIDS2017. Tấn công đối kháng (adversarial/evasion) nằm ngoài phạm vi đánh giá.

Kết quả tiền xử lý (tất định)

Nhóm nhãn mâu thuẫn bị loại	270 nhóm
Số dòng bị loại theo	44.260
Đặc trưng số còn lại	77
Huấn luyện / kiểm tra	1,79 tr / 0,45 tr

Chống thiên lệch lựa chọn

- Tách *pha thăm dò* (quét *K*) khỏi *pha xác nhận* — 4 cấu hình cố định *trước* khi xem kết quả.
- Mô hình đại diện chọn theo F1 trên **tập kiểm định** tách từ tập huấn luyện.
- Tập kiểm tra **không tham gia** bất kỳ bước lựa chọn nào.
- Siêu tham số cố định, dùng chung cho mọi phương pháp.

Nhóm lưỡng giống hết nhau trên mọi cột đặc trưng nhưng *nhãn xung đột* (lỗi đã ghi nhận của CICIDS2017) bị loại **cả nhóm** thay vì giữ lại một dòng tùy cách Spark phân vùng — nhờ vậy kết quả là tất định, tái lập được.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

↳ Q2 — Chi tiết tiền xử lý và chống thiên lệch lựa chọn

Q2 — Chi tiết tiền xử lý và chống thiên lệch lựa chọn

Kết quả tiền xử lý (tất định)	
Nhóm nhãn mâu thuẫn bị loại	270 nhóm
Số dòng bị loại theo	44.260
Đặc trưng số còn lại	77
Huấn luyện / kiểm tra	1,79 tr / 0,45 tr

Chống thiên lệch lựa chọn

- Tách *pha thăm dò* (quét *K*) khỏi *pha xác nhận* — 4 cấu hình cố định *trước* khi xem kết quả.
- Mô hình đại diện chọn theo F1 trên **tập kiểm định** tách từ tập huấn luyện.
- Tập kiểm tra **không tham gia** bất kỳ bước lựa chọn nào.
- Siêu tham số cố định, dùng chung cho mọi phương pháp.

Nhóm lưỡng giống hết nhau trên mọi cột đặc trưng nhưng *nhãn xung đột* (lỗi đã ghi nhận của CICIDS2017) bị loại **cả nhóm**, thay vì giữ lại một dòng tùy cách Spark phân vùng — nhờ vậy kết quả là tất định, tái lập được.

PCA $k=40$ thay vì $k=30$

PCA là phép *trích xuất*, không phải *chọn lọc*: mỗi thành phần chính là một tổ hợp tuyến tính chỉ mang một phần phương sai, nên cần nhiều thành phần hơn để giữ lượng thông tin tương đương tập 30 đặc trưng gốc. Trong dải quét chung $\{20, 30, 40\}$, $k=40$ là mức giữ phương sai cao nhất, nên là điểm vận hành đại diện *công bằng nhất* cho PCA; ép $k=30$ sẽ làm PCA mất thêm phương sai và bất lợi khi so sánh.

LightGBM bị loại khỏi so sánh

Thư viện native của LightGBM (qua SynapseML) chỉ hỗ trợ kiến trúc x86_64, không chạy được trên thiết bị biên Jetson Orin Nano Super (ARM64) — vốn là *đích triển khai* của luận văn. Trong họ gradient boosting, XGBoost và GBT đóng vai trò đại diện.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

└─ Q3 — Vì sao PCA dùng $k=40$, và vì sao loại LightGBM?

PCA $k=40$ thay vì $k=30$

PCA là phép *trích xuất*, không phải *chọn lọc*: mỗi thành phần chính là một tổ hợp tuyến tính chỉ mang một phần phương sai, nên cần nhiều thành phần hơn để giữ lượng thông tin tương đương tập 30 đặc trưng gốc. Trong dải quét chung $\{20, 30, 40\}$, $k=40$ là mức giữ phương sai cao nhất, nên là điểm vận hành đại diện *công bằng nhất* cho PCA; ép $k=30$ sẽ làm PCA mất thêm phương sai và bất lợi khi so sánh.

LightGBM bị loại khỏi so sánh

Thư viện native của LightGBM (qua SynapseML) chỉ hỗ trợ kiến trúc x86_64, không chạy được trên thiết bị biên Jetson Orin Nano Super (ARM64) — vốn là *đích triển khai* của luận văn. Trong họ gradient boosting, XGBoost và GBT đóng vai trò đại diện.

Q4 — Siêu tham số cố định trước (a priori)

Mô hình	Siêu tham số chính
Decision Tree	maxDepth=15, minInstancesPerNode=5, impurity=entropy
Logistic Regression	maxIter=200, regParam=0.001, L2, threshold=0.5
SVM (LinearSVC)	maxIter=200, regParam=0.001
Naive Bayes	gaussian, smoothing=1.0
Random Forest	numTrees=200, maxDepth=15, featureSubset=sqrt
GBT	maxIter=150, maxDepth=6, stepSize=0.05, subsample=0.8
XGBoost	n_estimators=300, max_depth=8, lr=0.05, subsample=0.8
MLP	layers=[d,64,32,2], maxIter=150, stepSize=0.01

Dùng chung cho mọi phương pháp giảm chiều, để khác biệt quan sát được phản ánh đúng *phương pháp chọn đặc trưng*. Tối ưu siêu tham số (grid search + cross-validation) tách thành giai đoạn riêng, chỉ áp dụng cho cấu hình đã chọn — mức cải thiện nhỏ, phù hợp với nhận định bài toán nhị phân trong miền đã gán bảo hòa.

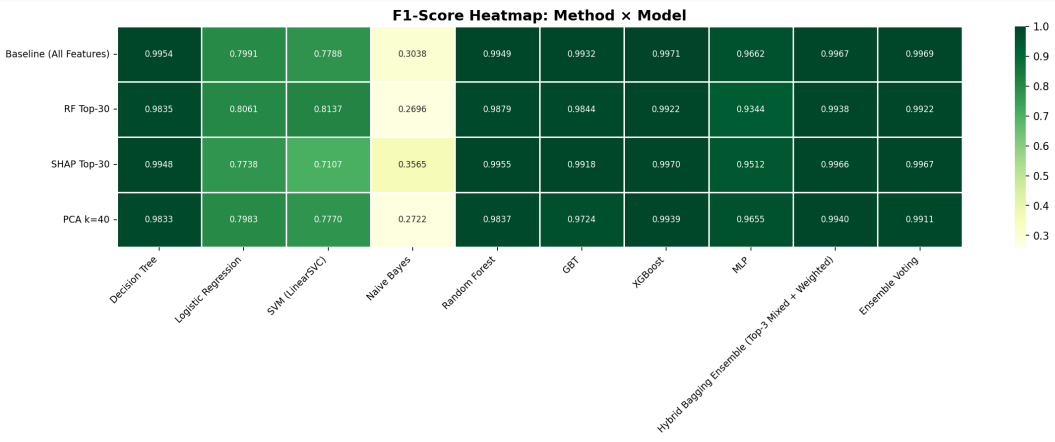
Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Q4 — Siêu tham số cố định trước (a priori)

Q4 — Siêu tham số cố định trước (a priori)	
Mô hình	Siêu tham số chính
Decision Tree	maxDepth=15, minInstancesPerNode=5, impurity=entropy
Logistic Regression	maxIter=200, regParam=0.001, L2, threshold=0.5
SVM (LinearSVC)	maxIter=200, regParam=0.001
Naive Bayes	gaussian, smoothing=1.0
Random Forest	numTrees=200, maxDepth=15, featureSubset=sqrt
GBT	maxIter=150, maxDepth=6, stepSize=0.05, subsample=0.8
XGboost	n_estimators=300, max_depth=8, lr=0.05, subsample=0.8
MLP	layers=[d,64,32,2], maxIter=150, stepSize=0.01

Dùng chung cho mọi phương pháp giảm chiều, để khác biệt quan sát được phản ánh đúng *phương pháp chọn đặc trưng*. Tối ưu siêu tham số (grid search + cross-validation) tách thành giai đoạn riêng, chỉ áp dụng cho cấu hình đã chọn — mức cải thiện nhỏ, phù hợp với nhận định bài toán nhị phân trong miền đã gán bảo hòa.

Q5 — Bản đồ nhiệt F1: phương pháp × thuật toán

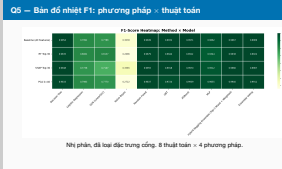


Nhị phân, đã loại đặc trưng cộng, 8 thuật toán × 4 phương pháp.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Q5 — Bản đồ nhiệt F1: phương pháp × thuật toán



- **Nội tại.** Xếp hạng RF/SHAP lấy từ tập huấn luyện gốc và giữ cố định qua các lần lấy mẫu lại — nhất quán với thiết kế “cấu hình cố định trước”, nhưng có thể gây rò rỉ nhẹ ở phần xếp hạng. Xếp hạng lại theo từng lần lấy mẫu chặt chẽ hơn nhưng rất tốn kém với SHAP. MLP không được gán trọng số lớp (Spark ML không hỗ trợ).
- **Ngoại tại.** CICIDS2017 (2017) có thể không phản ánh lưu lượng hiện tại; các bộ dữ liệu NIDS chuẩn đều có hạn chế chất lượng đã ghi nhận. Kết luận liên bộ dữ liệu dựa trên *mẫu phân tầng* của CSE-CIC-IDS2018. Chưa đánh giá trước lưu lượng đối kháng. Bản thân việc khử trùng lặp cũng làm thay đổi phân bố benign/attack.
- **Thống kê.** $N=6$ cho *power* thấp: kiểm định hai phía liệt kê đầy đủ chặn p ở mức $2/2^N$, nên $N=6$ chỉ vừa đủ chạm ngưỡng 0,05. Các lần lấy mẫu chồng lấn, vi phạm giả định độc lập của phân phối t — do đó Nadeau–Bengio là căn cứ chính. Tăng N với cross-validation lồng nhau là hướng củng cố tiếp theo.

Q7 — Vì sao chọn Random Forest cho thiết bị biên?

Cấu hình	F1	Đặc trưng	Dung lượng (MB)	Lý do ở pha biên
Baseline + XGBoost	0,9971	77	2,32	Mắc tham chiếu
SHAP Top-30 + XGBoost	0,9970	30	0,003 ¹	Giải thích được
RF Top-30 + XGBoost	0,9922	30	2,52	Xếp hạng nhúng
PCA k=40 + XGBoost	0,9939	40	3,82	Giảm chiều không giám sát

Mô hình triển khai là **SHAP Top-30 với Random Forest** (F1 offline 0,9955, chênh 0,002 so với XGBoost tốt nhất): Random Forest là estimator gốc của Spark ML nên PipelineModel xuất ra nạp được trên board ARM64 mà không cần thư viện native mà tích hợp XGBoost của Spark đòi hỏi. Vector đầu vào có **29** đặc trưng — destination_port nằm trong xếp hạng SHAP nhưng bị loại vì rò rỉ nhãn.

¹Bước lưu mô hình phân tán chỉ ghi metadata cục bộ trên driver nên bảo thiếu dung lượng thật (~2,5 MB).

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Q7 — Vì sao chọn Random Forest cho thiết bị biên?

Q7 — Vì sao chọn Random Forest cho thiết bị biên?

Cấu hình	F1	Đặc trưng	Dung lượng (MB)	Lý do ở pha biên
Baseline + XGBoost	0,9971	77	2,32	Mắc tham chiếu
SHAP Top-30 + XGBoost	0,9970	30	0,003 ¹	Giải thích được
RF Top-30 + XGBoost	0,9922	30	2,52	Xếp hạng nhúng
PCA k=40 + XGBoost	0,9939	40	3,82	Giảm chiều không giám sát

Mô hình triển khai là SHAP Top-30 với Random Forest (F1 offline 0,9955, chênh 0,002 so với XGBoost tốt nhất): Random Forest là estimator gốc của Spark ML nên PipelineModel xuất ra nạp được trên board ARM64 mà không cần thư viện native mà tích hợp XGBoost của Spark đòi hỏi.

Vector đầu vào có **29** đặc trưng — destination_port nằm trong xếp hạng SHAP nhưng bị loại vì rò rỉ nhãn.

¹Bước lưu mô hình phân tán chỉ ghi metadata cục bộ trên driver nên bảo thiếu dung lượng thật (~2,5 MB).

Q8 — Ba chế độ triển khai và môi trường đo

Đơn nút toàn bộ pipeline trong một tiến trình — mốc tham chiếu.

A: Tách pipeline cổng trên #1, phân lớp trên #2 (**thiết kế đề xuất**).

B: Mở rộng ngang cả hai nút chạy *vai trò đầy đủ*, chia tải qua Kafka consumer group.

C: Cụm Spark máy chủ làm master, hai Jetson làm worker.

Thông lượng đến: **số luồng xử lý xong mỗi giây** — Mỗi luồng chỉ tính một lần, tại nơi có kết luận ở cổng nếu bị bỏ qua, ở bộ phân lớp nếu được chuyển tiếp — nhờ vậy luồng đi qua cả hai nút không bị đếm hai lần.

Môi trường thực nghiệm

2× Jetson Orin Nano Super: CPU 6 nhân Arm Cortex-A78AE, GPU Ampere ≈67 TOPS, 8 GB LPDDR5, SSD NVMe 256 GB.

PySpark 3.4.1, Kafka 3.5, LAN Wi-Fi.

Tải: phát lại CICIDS2017 ở 100 luồng/giây.

≥3 lần lặp; lưu lượng khởi động *loại khỏi* cửa sổ đo.

Năng lượng: tegrastats, trừ nền idle.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Q8 — Ba chế độ triển khai và môi trường đo

Cách đếm “luồng xử lý xong/giây” là điểm mấu chốt để so sánh công bằng giữa các chế độ: nếu đếm theo luồng đi qua mỗi nút thì chế độ tách pipeline sẽ được cộng đôi một cách vô lý.

Q8 — Ba chế độ triển khai và môi trường đo		
Đơn nút	toàn bộ pipeline trong một tiến trình — mốc tham chiếu.	Môi trường thực nghiệm 2× Jetson Orin Nano Super: CPU 6 nhân Arm Cortex-A78AE, GPU Ampere ≈67 TOPS, 8 GB LPDDR5, SSD NVMe 256 GB. PySpark 3.4.1, Kafka 3.5, LAN Wi-Fi. Tải: phát lại CICIDS2017 ở 100 luồng/giây. ≥3 lần lặp; lưu lượng khởi động loại khỏi cửa sổ đo. Năng lượng: tegrastats, trừ nền idle.
	A: Tách pipeline cổng trên #1, phân lớp trên #2 (thiết kế đề xuất).	
B: Mở rộng ngang	cả hai nút chạy vai trò đầy đủ, chia tải qua Kafka consumer group.	
	C: Cụm Spark máy chủ làm master, hai Jetson làm worker.	

- Script điều phối phía máy chủ khởi động các vai trò pipeline trên Jetson qua SSH, sinh tải, thu thập log từng nút.
- ≥ 3 lần lặp mỗi cấu hình; lưu lượng khởi động *loại khỏi* cửa sổ đo, để JVM/JIT warmup không làm phồng độ trễ batch đầu.
- Mọi truy vấn chỉ số canh đúng cửa sổ tải, không dùng khoảng thời gian trượt phía sau.
- Phân vị độ trễ tính *theo từng nút* trên mẫu thô từng luồng; p95 mức cụm lấy *thận trọng* theo nút tệ nhất.
- Độ trễ đầu cuối dựa trên đồng hồ đồng bộ NTP và *không* bao gồm chi phí ghi cơ sở dữ liệu.
- Năng lượng: tegrastats, nền idle 30 giây rồi đến cửa sổ tải; chế độ hai nút cộng công suất cả hai board.

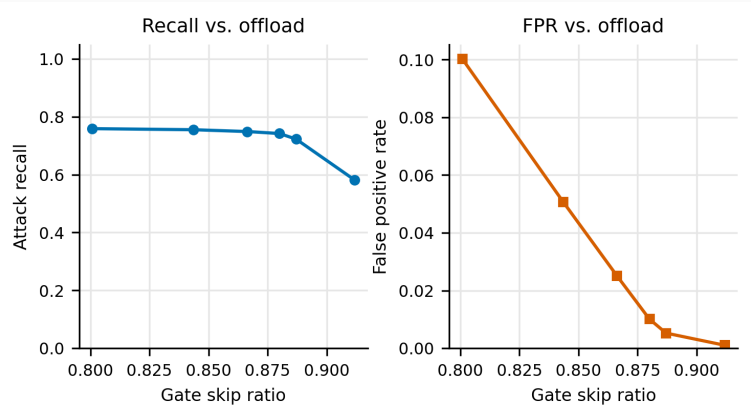
2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Q9 — Giao thức đo hiệu năng ở pha biên

- Script điều phối phía máy chủ khởi động các vai trò pipeline trên Jetson qua SSH, sinh tải, thu thập log từng nút.
- ≥ 3 lần lặp mỗi cấu hình; lưu lượng khởi động *loại khỏi* cửa sổ đo, để JVM/JIT warmup không làm phồng độ trễ batch đầu.
- Mọi truy vấn chỉ số canh đúng cửa sổ tải, không dùng khoảng thời gian trượt phía sau.
- Phân vị độ trễ tính theo từng nút trên mẫu thô từng luồng; p95 mức cụm lấy *thận trọng* theo nút tệ nhất.
- Độ trễ đầu cuối dựa trên đồng hồ đồng bộ NTP và không bao gồm chi phí ghi cơ sở dữ liệu.
- Năng lượng: tegrastats, nền idle 30 giây rồi đến cửa sổ tải; chế độ hai nút cộng công suất cả hai board.

Q10 — Cổng bất thường là một *đường cong vận hành*

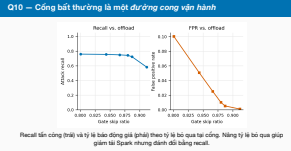


Recall tấn công (trái) và tỷ lệ báo động giả (phải) theo tỷ lệ bỏ qua tại cổng. Năng tỷ lệ bỏ qua giúp giảm tải Spark nhưng đánh đổi bằng recall.

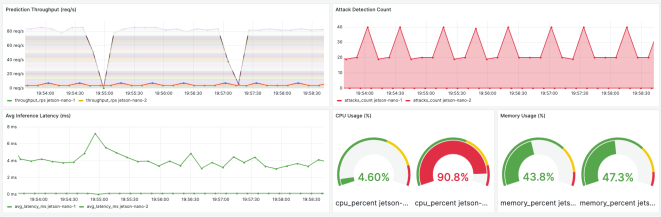
2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Q10 — Cổng bất thường là một *đường cong vận hành*



Q11 — Bất đối xứng tải lúc đỉnh tải (Grafana)



Tại đỉnh tải, chế độ tách pipeline:

- nút cổng: **4,6% CPU**
- nút phân lớp: **90,8% CPU**

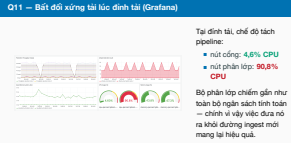
Bộ phân lớp chiếm gần như toàn bộ ngân sách tính toán — chính vì vậy việc đưa nó ra khỏi đường ingest mới mang lại hiệu quả.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Q11 — Bất đối xứng tải lúc đỉnh tải (Grafana)

Ảnh chụp Grafana thật, không phải sơ đồ. Hệ thống giám sát thời gian thực: thông lượng, số tấn công phát hiện, độ trễ, CPU/RAM từng nút, cảnh báo lưu trong PostgreSQL.



Tại đỉnh tải, chế độ tách pipeline:
■ nút cổng: **4,6% CPU**
■ nút phân lớp: **90,8% CPU**
Bộ phân lớp chiếm gần như toàn bộ ngân sách tính toán — chính vì vậy việc đưa nó ra khỏi đường ingest mới mang lại hiệu quả.

- **Cải thiện độ nhạy cho lớp tấn công hiếm.** Xử lý mất cân bằng có chủ đích (undersampling/SMOTE áp dụng *sau* khi chia tập), focal loss, hạ ngưỡng quyết định theo lớp; **kiến trúc phân tầng** — tận dụng chính thiết kế cổng hai tầng: tầng nhị phân ở biên giữ recall tổng, còn bộ định danh loại chỉ huấn luyện trên *luồng tấn công*, huấn luyện lại ngoại tuyến mà không đụng hệ thống đang chạy; thêm **đặc trưng chuỗi thời gian theo host** cho beaconing C&C.
- **Kiểm chứng trên dữ liệu IoT chuyên biệt.** Bot-IoT, TON_IoT, CIC-IoT-2023; mở rộng thí nghiệm liên bộ dữ liệu lên toàn bộ CSE-CIC-IDS2018.
- **Tối ưu suy luận biên bằng ONNX/TensorRT.** Tích hợp phiên suy luận ONNX vào pipeline streaming, tăng tốc bằng TensorRT trên GPU Jetson; đo độ trễ đầu cuối đầy đủ.
- **Bền vững trước tấn công đối kháng.** Đặc biệt hai bề mặt tấn công mới do cổng tạo ra: giả dạng lưu lượng bình thường để vượt cổng, và cố ý tạo sai số tái tạo cao để dồn tải lên nút phân lớp.

2026-08-15

Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Q12 — Hướng phát triển chi tiết

Mỗi hướng phát triển ứng với một hạn chế ở slide trước — đó là cách thể hiện sự làm chủ đề tài.

Q12 — Hướng phát triển chi tiết

- **Cải thiện độ nhạy cho lớp tấn công hiếm.** Xử lý mất cân bằng có chủ đích (undersampling/SMOTE áp dụng *sau* khi chia tập), focal loss, hạ ngưỡng quyết định theo lớp; **kiến trúc phân tầng** — tận dụng chính thiết kế cổng hai tầng: tầng nhị phân ở biên giữ recall tổng, còn bộ định danh loại chỉ huấn luyện trên *luồng tấn công*, huấn luyện lại ngoại tuyến mà không đụng hệ thống đang chạy; thêm **đặc trưng chuỗi thời gian theo host** cho beaconing C&C.
- **Kiểm chứng trên dữ liệu IoT chuyên biệt.** Bot-IoT, TON_IoT, CIC-IoT-2023; mở rộng thí nghiệm liên bộ dữ liệu lên toàn bộ CSE-CIC-IDS2018.
- **Tối ưu suy luận biên bằng ONNX/TensorRT.** Tích hợp phiên suy luận ONNX vào pipeline streaming, tăng tốc bằng TensorRT trên GPU Jetson; đo độ trễ đầu cuối đầy đủ.
- **Bền vững trước tấn công đối kháng.** Đặc biệt hai bề mặt tấn công mới do cổng tạo ra: giả dạng lưu lượng bình thường để vượt cổng, và cố ý tạo sai số tái tạo cao để dồn tải lên nút phân lớp.